PurpleDropTaxi — Booking Engine

Google Maps API Setup Process + Fare Estimation Logic

This document covers two things for the PurpleDropTaxi Next.js frontend: (1) the exact steps to make the Google Maps integration work in production (what's already wired in code vs. what needs to be configured in Google Cloud Console), and (2) how the estimated fare shown to a customer is actually calculated, end to end.

Part 1 — Google Maps API Setup Process

1.1 What the app needs from Google Maps

The booking form uses three distinct Google services, all through one API key:

- **Maps JavaScript API** — renders the live map and route on the /estimate page.
- **Places API** — powers the Pickup/Drop location autocomplete suggestions.
- **Geocoding API** — resolves free-typed text (when a customer doesn't click a suggestion) into latitude/longitude, so a route can still be calculated.
- **Directions API** — computes the driving distance and duration between pickup and drop, which feeds directly into the fare calculation.

1.2 Where it's wired in the code

The key is read from a single environment variable and used consistently across both pages:

```
// .env.local (create this file, never commit it)
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=your_key_here

// app/lib/booking.js
export const GOOGLE_MAPS_LIBRARIES = ["places"];

// app/Components/BookingSection.js  (page 1 — form)
useJsApiLoader({
  id: "google-map-script",
  googleMapsApiKey: process.env.NEXT_PUBLIC_GOOGLE_MAPS_API_KEY ?? "",
  libraries: GOOGLE_MAPS_LIBRARIES,
});
// -> <Autocomplete> on Pickup/Drop inputs
// -> new google.maps.Geocoder() as a fallback when no suggestion is clicked

// app/estimate/page.js  (page 2 — map + fare + confirm)
// -> <GoogleMap> + <DirectionsRenderer> to draw the route
// -> new google.maps.DirectionsService() to compute distanceKm / durationMins
```

The env var must be prefixed `NEXT_PUBLIC_` — Next.js only exposes variables with that prefix to client-side code, which is required since this key runs in the browser.

1.3 Google Cloud Console setup (step by step)

- Go to **console.cloud.google.com** and create (or select) a project.

- **APIs & Services -> Enabled APIs** — enable all four: Maps JavaScript API, Places API, Geocoding API, Directions API.
- **Billing** — link a billing account to the project. This is required even to use the free monthly credit; without it every request is rejected outright.
- **APIs & Services -> Credentials** — create an API key (or use an existing one).
- Under the key's **Application restrictions**, add the domains that will call it: your production domain, and `localhost` (or `http://localhost:3000/*`) for local development.
- Under **API restrictions**, restrict the key to only the four APIs above (good practice, not required).
- Paste the key into `.env.local` as shown above and restart the dev server.

1.4 Verifying it actually works

A key can look configured but still silently fail. The fastest way to check is directly from the browser console on the booking form page:

```
// Paste in the browser DevTools console while on the booking page
new google.maps.places.AutocompleteService().getPlacePredictions(
  { input: "Pollachi" },
  (predictions, status) => console.log(status, predictions)
);

new google.maps.Geocoder().geocode(
  { address: "Pollachi, Tamil Nadu" },
  (results, status) => console.log(status, results)
);
```

Reading the status:

- **`OK`** — working correctly, results returned.
- **`ZERO_RESULTS`** — API is enabled and billed, the query just didn't match anything.
- **`REQUEST_DENIED`** — the API itself is rejecting the request. Almost always one of: billing not enabled, the specific API not enabled, or the calling domain isn't in the key's referrer restrictions.

This exact diagnostic was used on this project and returned `REQUEST_DENIED` for both Places and Geocoding, confirming the key needs billing/API enablement before location search or route calculation can work.

1.5 What happens if the key isn't working yet

The app is built to degrade gracefully rather than break the booking flow:

- Pickup/Drop inputs fall back to plain free-text fields (no autocomplete dropdown).
- On submit, the app still attempts to geocode whatever text was typed; if that also fails, the booking proceeds anyway with no coordinates attached.
- On the estimate page, the map shows a placeholder instead of live tiles, and the route/distance/duration display is simply omitted.
- The fare estimate still generates — see Part 2 for exactly what it falls back to when no real distance is available.

Part 2 — Fare (Price) Estimation Process

2.1 Where distance and duration come from

Pickup and drop are first resolved to map coordinates (lat/lng) — either from clicking a Places Autocomplete suggestion, selecting a fixed airport location, or the geocoding fallback on submit. On the /estimate page, those two coordinates are sent to Google's Directions Service:

```
// app/estimate/page.js
directionsService.route(
  { origin: pickupCoords, destination: dropCoords,
    travelMode: google.maps.TravelMode.DRIVING },
  (result, status) => {
    const leg = result.routes[0].legs[0];
    distanceKm  = round(leg.distance.value / 1000, 1);
    durationMins = round(leg.duration.value / 60);
  }
);
```

If either coordinate is missing (Maps key not working, or geocoding failed), this step is skipped entirely and `distanceKm` defaults to `0` for the calculation below.

2.2 Vehicle rate card

Vehicle	Rate per km	Seats
Sedan	Rs. 15 / km	4
Etios	Rs. 16 / km	4
SUV	Rs. 20 / km	6
Prime SUV	Rs. 22 / km	6

Defined once in `app/lib/booking.js` (`FALLBACK_VEHICLES`) and shared by both pages, so the vehicle-select cards on the form and the fare breakdown on the estimate page always agree.

2.3 The calculation, step by step

Implemented in `apiEstimateFare()`, `app/lib/booking.js`:

```
distance = tripType === "roundtrip" ? distanceKm * 2 : distanceKm

base          = distance * vehicle.ratePerKm
driverBata    = 400                                  (flat, every trip)
tollCharges   = distance > 80 ? 120 : 0               (long trips only)
hillCharges   = 0                                     (not currently used)
gst           = round( (base + driverBata + tollCharges) * 0.05 )

fare = round( base + driverBata + tollCharges + hillCharges + gst )
```

2.4 Worked example

Input	Value

Trip type	One Way
Vehicle	SUV (Rs. 20/km)
Distance (from Directions API)	90 km

Step	Amount
Base fare (90 km x Rs. 20)	Rs. 1,800
Driver bata (flat)	Rs. 400
Toll charges (distance > 80 km)	Rs. 120
Hill charges	Rs. 0
GST (5% of 1,800 + 400 + 120)	Rs. 116
Total estimated fare	**Rs. 2,436**

> For a Round Trip, the same 90 km would be doubled to 180 km before the base-fare step, changing every downstream number.

2.5 Fallback when no distance is available

If the Maps key isn't working yet (see Part 1) and no distance was computed, `distanceKm` is `0`. The formula still runs, so the customer sees a fare of just **driver bata + GST on that bata** (Rs. 400 + Rs. 20 GST = **Rs. 420**) regardless of the real trip — a placeholder number, not a real quote. This is why getting the Maps API key working (Part 1) is required before real fares can be shown.

2.6 Where the enquiry goes after this

Once the customer taps **Confirm Booking**, the fare + form data is sent to `apiSubmitEnquiry()`. A real backend endpoint (`POST /api/enquiry`) is not live yet — until it is, the app mocks a successful response locally (with a generated `ENQ-######` reference number) purely so the UI flow can be demoed end to end. This is not a live booking system: a human tele-calling team is expected to call the customer to confirm the ride.
